

# Data Science

---

## Lecture 03: Probabilistic Graphical Models

Prof. Dr. Michael Granitzer

# Where We Left Off

Lecture 02 gave us conditioning and Bayes for a handful of events. Real questions involve dozens of variables at once — and the assumption that makes them affordable says *which variables do not depend on each other*.

## Lecture 02

Bayes for two events



## Today

one joint distribution over many variables

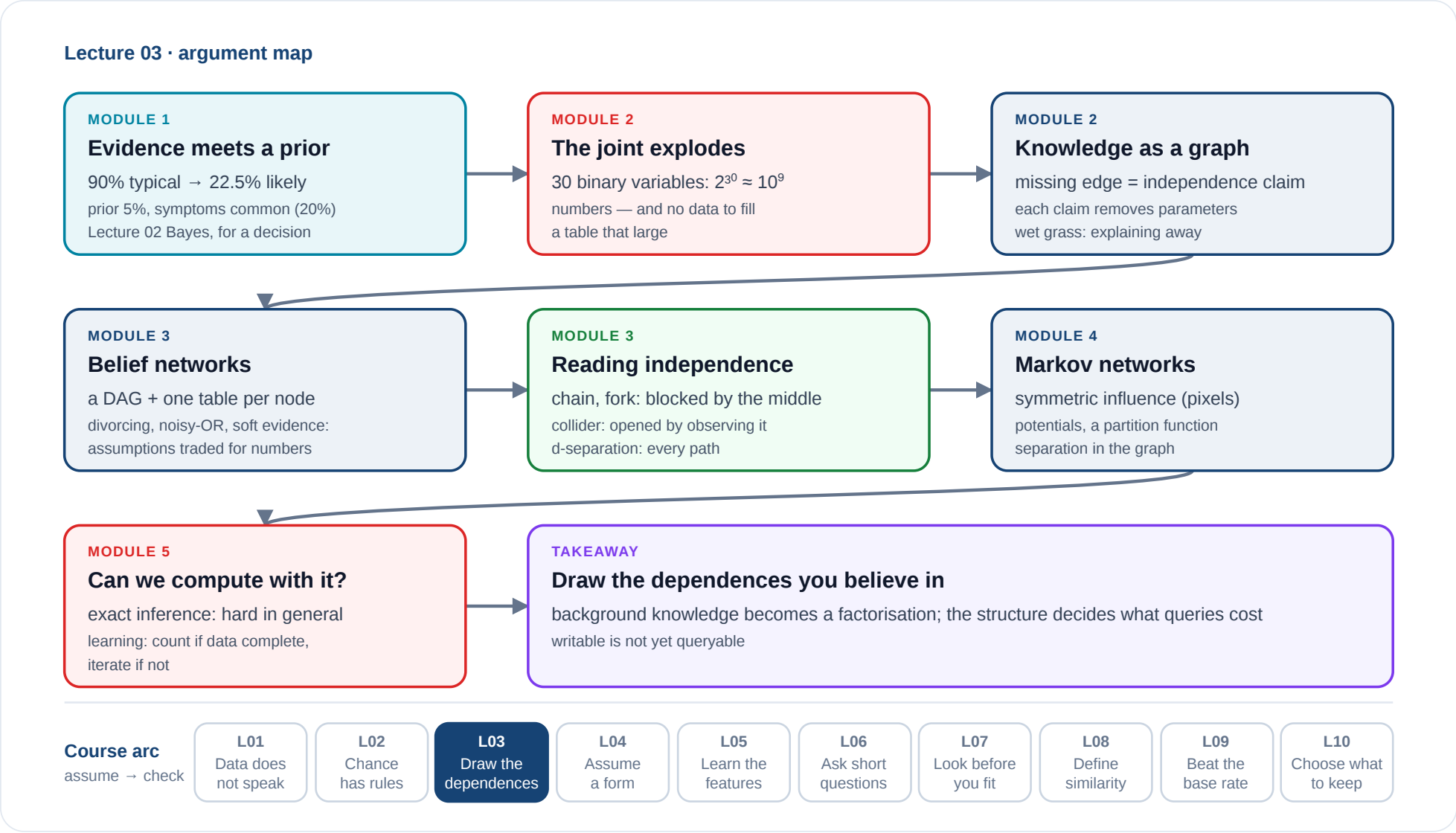


## The assumption

a graph: every missing edge an independence claim

Lecture 01's churn puzzle — does the app keep customers, or do loyal customers install the app? — is a question about arrows. Module 3 draws both answers.

# Argument Map



# Module 1

---

## Deciding Under Uncertainty

**Guiding Question:** *If a symptom is 90% typical of a disease, why does having the symptom not make the disease 90% likely?*

# Thinking With Uncertainty

Bayes' theorem lets us update a **hypothesis**  $H$  in the light of **data**  $D$ :

$$P(H \mid D) = \frac{P(H) \cdot P(D \mid H)}{P(D)}$$

This is the whole apparatus of evidence-based decision making in one line: a belief, a measurement of how well the belief explains what we saw, and a normalisation.

# Example: Do I Have the Flu?

You have a headache and a cough. From the literature:

## Likelihood

90% of people with the flu  
show headache and coughing

$$P(D \mid H) = 0.9$$

## Prior

5% of people currently have  
the flu

$$P(H) = 0.05$$

## Evidence

headache and coughing occur  
in 20% of people

$$P(D) = 0.2$$

**How likely is it that you have the flu?**

# Example: Do I Have the Flu?

You have a headache and a cough. From the literature:

## Likelihood

90% of people with the flu  
show headache and coughing

$$P(D \mid H) = 0.9$$

## Prior

5% of people currently have  
the flu

$$P(H) = 0.05$$

## Evidence

headache and coughing occur  
in 20% of people

$$P(D) = 0.2$$

How likely is it that you have the flu?

$$P(H \mid D) = \frac{0.05 \cdot 0.9}{0.2} = 0.225$$

# Why So Low?

## Few people have the flu

The prior  $P(H) = 0.05$  caps how far the evidence can carry us.

## The symptoms are common

$P(D) = 0.2$  means many people have these symptoms *without* the flu.

A symptom is informative to the degree that it is **rare in general** but **common under the hypothesis**. A likelihood alone tells you nothing.

# Takeaway

Bayesian decision making weighs **prior × likelihood** against how ordinary the evidence is. Deciding under uncertainty means keeping all three numbers in view, not just the one the study reports.

# Module 2

---

## Why Graphical Models?

**Guiding Question:** *Thirty binary variables need a billion numbers. What knowledge would let us write the same distribution with a few dozen?*

# The Problem: Joint Distributions Explode

Consider a joint distribution  $p(x_1, \dots, x_n)$  over  $n$  **binary** random variables. How many configurations are there?

# The Problem: Joint Distributions Explode

Consider a joint distribution  $p(x_1, \dots, x_n)$  over  $n$  **binary** random variables. How many configurations are there?

$2^n$  states,  $2^n - 1$  free parameters

## Specification

For large  $n$ , writing down all values is impossible.

## Inference

A single marginal  $p(x_1)$  requires summing over  $2^{n-1}$  states.

## Chain rule?

It rewrites the joint exactly — but does not reduce the number of values.

# What a Graphical Model Is

**Definition (Graphical model).** A graphical model depicts the dependence and independence relations of a distribution using a graph  $G$  as its underlying structure.

## Modelling

Describe the variables of a domain and their interactions — how  $p(x_1, \dots, x_n)$  **factorises**.

## Inference

Answer queries such as  $p(x_1 \mid x_2 = a, \dots)$  efficiently over that factorisation.

Usually one class of model is used for modelling and another for inference; the latter is often compiled from the former.

# How a Graph Can Help

Probabilistic inference is summing and multiplying. Let every variable be a node, and read edges as interaction:

## No edge

$$f(a, b) = g(a) \cdot h(b)$$

The variables are independent — the factor splits.

## Edge

$$f(a, b) \neq g(a) \cdot h(b)$$

The variables are dependent — the factor does not split.

Edge **direction** can encode further structure. The principle extends to any number of variables.

Graphical models have limited expressive power: not every conditional independence statement can be drawn, and different graphs can represent the same distribution.

# Example: Why Is the Grass Wet?

Tracey leaves her house and sees that her grass is wet. **Was it rain, or did she leave the sprinkler on?** She notices her neighbour Jack's grass is also wet.

*R*

raining (1) or not (0)

*S*

Tracey's sprinkler on  
(1) or off (0)

*J*

Jack's grass wet (1)  
or dry (0)

*T*

Tracey's grass wet  
(1) or dry (0)

# Building the Model

The full joint over four binary variables needs  $2^4 - 1 = 15$  probabilities. The chain rule rewrites it but does not shrink it:

$$p(T, J, R, S) = p(T \mid J, R, S)p(J \mid R, S)p(R \mid S)p(S)$$

Now bring in **background knowledge** — assumptions of conditional independence:

Tracey's grass does not depend on Jack's

$$p(T \mid J, R, S) = p(T \mid R, S)$$

Jack's grass does not depend on Tracey's sprinkler

$$p(J \mid R, S) = p(J \mid R)$$

Rain is not influenced by the sprinkler

$$p(R \mid S) = p(R)$$

$$p(T, J, R, S) = p(T \mid R, S)p(J \mid R)p(R)p(S)$$

Only  $4 + 2 + 1 + 1 = 8$  probabilities remain — and each one is a question a person can actually answer.

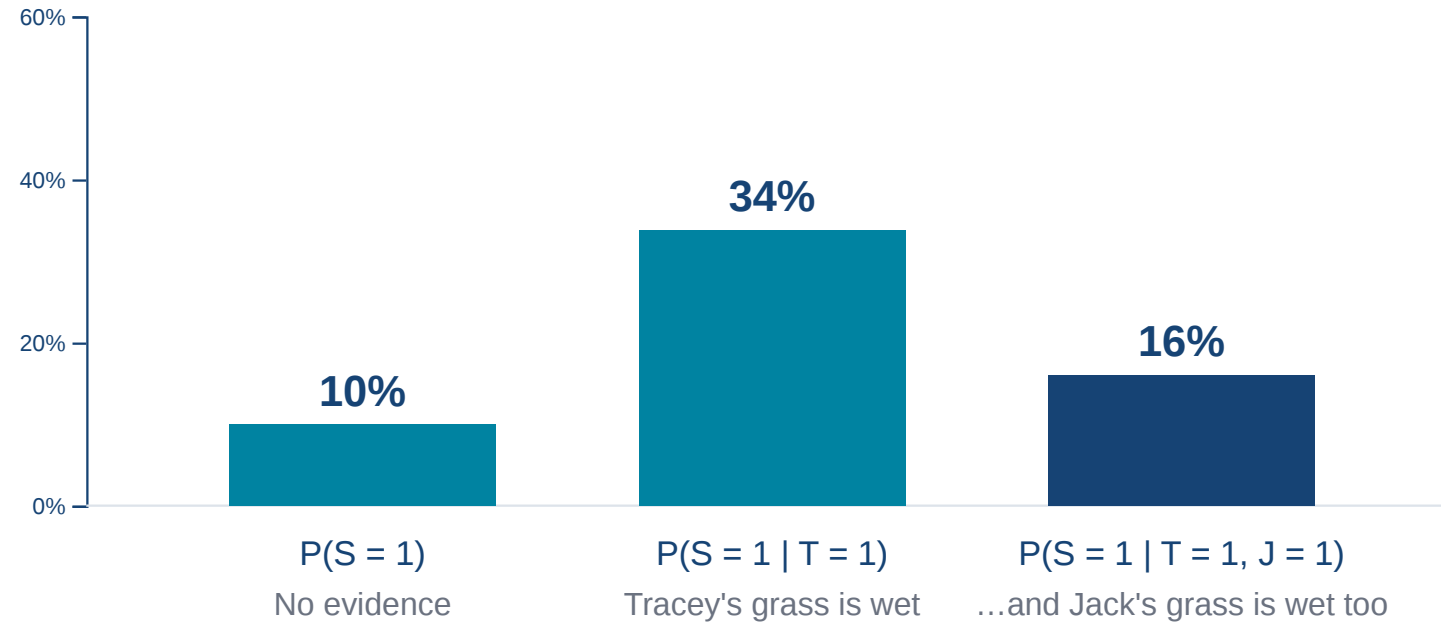
# The Probability Tables

| Factor                       | $p(\cdot)$ | $1 - p(\cdot)$ |
|------------------------------|------------|----------------|
| $p(R = 1)$                   | 0.2        | 0.8            |
| $p(S = 1)$                   | 0.1        | 0.9            |
| $p(J = 1 \mid R = 0)$        | 0.2        | 0.8            |
| $p(J = 1 \mid R = 1)$        | 1.0        | 0.0            |
| $p(T = 1 \mid R = 1, S = 0)$ | 1.0        | 0.0            |
| $p(T = 1 \mid R = 1, S = 1)$ | 1.0        | 0.0            |
| $p(T = 1 \mid R = 0, S = 1)$ | 0.9        | 0.1            |
| $p(T = 1 \mid R = 0, S = 0)$ | 0.0        | 1.0            |

Missing edges in the graph are exactly the assumptions above; the tables supply the numbers.

# Inference: What Does the Evidence Say?

Rain explains the wetness — the sprinkler falls back towards its prior



# Explaining Away

$$p(S = 1 \mid T = 1, J = 1) = \frac{\sum_R p(T=1 \mid R, S=1)p(J=1 \mid R)p(R)p(S=1)}{\sum_{R,S} p(T=1 \mid R, S)p(J=1 \mid R)p(R)p(S)} = \frac{0.0344}{0.2144} \approx 0.16$$

Two independent causes ( $R$  and  $S$ ) become **dependent once their common effect is observed**. Evidence for rain — Jack's wet grass — lowers the probability of the sprinkler from 0.34 to 0.16, even though rain and sprinkler are unconditionally independent.

# Takeaway

A graphical model turns **background knowledge into a factorisation**: missing edges are conditional independence assumptions, and each one removes parameters. The graph is a picture of how the joint splits — nothing more, and nothing less.

# Module 3

---

## Bayesian Belief Networks

**Guiding Question:** *What exactly does an arrow in a probabilistic model claim — and what does a missing arrow claim?*

# Definition

**Bayesian belief network (BBN).** A **directed acyclic graph** whose nodes are random variables, in which the joint distribution factorises as

$$p(x_1, \dots, x_n) = \prod_{i=1}^n p(x_i \mid \text{pa}(x_i))$$

where  $\text{pa}(x_i)$  are the parents of  $x_i$  in the graph.

Each node carries one conditional probability table given its parents. The wet-grass model was exactly this.

**What the graph claims:** each variable is independent of its non-descendants given its parents. A *missing* arrow is an independence claim; an arrow only permits a dependence.

# Three Tasks

## Modelling

Choose variables, edges and tables that encode the domain.

## Inference

Compute posteriors given evidence — at a cost set by the graph's shape.

## Learning

Estimate the tables — and possibly the structure — from data.

# Modelling Technique: Divorcing Parents

A node with  $k$  binary parents needs  $2^k$  table entries. **Divorcing** inserts an intermediate variable that summarises a group of parents, so the table becomes two small ones instead of one large one.

The intermediate node usually has a meaning of its own — "total load", "combined risk" — which is why the technique improves interpretability as well as size.

Fewer parameters means fewer numbers a domain expert has to invent.

# Modelling Technique: Noisy OR

When many causes  $x_1, \dots, x_k$  can each independently trigger an effect  $y$ , a **noisy OR** models the effect as "each cause fires with its own probability, and the effect occurs if any of them fires":

$$p(y = 0 \mid x_1, \dots, x_k) = \prod_{i: x_i=1} (1 - \lambda_i)$$

$k$  parameters instead of  $2^k$ .

## Typical use

Disease–symptom networks, where many diseases give rise to the same symptom.

# Uncertain and Unreliable Evidence

## Uncertain (soft) evidence

The state of  $x$  is itself uncertain, expressed as a distribution  $\tilde{x}$ . Infer by summing over the states:

$$p(y \mid \tilde{x}) = \sum_x p(y \mid x)p(x \mid \tilde{x})$$

*We look at the grass without touching it:*

$$p(T=1 \mid \tilde{T}) = 0.8.$$

## Unreliable evidence

The report is kept, but its table is **replaced** by our own judgement of how reliable it is:

$$p(x \mid \text{pa}(x)) \rightarrow \tilde{p}(x \mid \text{pa}(x))$$

*Jack says his grass is wet, but Jack is in a bad mood and did not check.*

Both are drawn with a **dashed** edge.

# Takeaway

A belief network is a **DAG plus one conditional table per node**. Modelling techniques — divorcing, noisy OR, soft evidence — all trade assumptions for parameters, and every trade should be stated out loud.

# Three Ways Three Nodes Can Connect

Every conditional-independence question in a DAG reduces to these three patterns.

## Chain

$$a \rightarrow c \rightarrow b$$

$a$  and  $b$  are dependent — but **independent given  $c$** .

*Information flows along the chain until the middle is observed.*

## Common cause (fork)

$$a \leftarrow c \rightarrow b$$

$a$  and  $b$  are dependent — but **independent given  $c$** .

*The shared cause explains the correlation.*

## Common effect (collider)

$$a \rightarrow c \leftarrow b$$

$a$  and  $b$  are **independent** — but **dependent given  $c$** .

*Explaining away — the reverse of the other two.*

# Mini-Exercise

Rain and the sprinkler are independent:  $\text{rain} \rightarrow \text{wet grass} \leftarrow \text{sprinkler}$ . Your grass is **wet**, and you learn that it **rained**.

Does that change how likely it is that the sprinkler was on?

# Mini-Exercise

Rain and the sprinkler are independent: rain  $\rightarrow$  wet grass  $\leftarrow$  sprinkler. Your grass is **wet**, and you learn that it **rained**.

Does that change how likely it is that the sprinkler was on?

**Yes — it drops.** Rain already explains the wet grass. Observing the common effect made two independent causes dependent: *explaining away*, the collider case. On a chain or a fork, observing the middle does the opposite.

# d-Separation

A path between  $X$  and  $Y$  is **blocked** by a set  $Z$  if it contains either

- a chain or fork whose middle node **is in**  $Z$ , or
- a collider whose middle node — and all of its descendants — are **not in**  $Z$ .

If every path from  $X$  to  $Y$  is blocked by  $Z$ , then  $X$  and  $Y$  are **d-separated** given  $Z$ , and  $X \perp\!\!\!\perp Y \mid Z$ .

d-separation is sound: what it declares independent really is independent for every distribution factorising over the graph. It is not complete for a *particular* distribution — extra independences may hold numerically without being visible in the graph.

# Takeaway

Independence in a belief network is a **graph property**: chains and forks are blocked *by* observing the middle node, colliders are opened by it. d-separation is that rule applied to every path.

# The Churn Shop, Drawn

App users churn less (Lecture 02: the gap is real). Two graphs explain it:

## FORK

**app**  $\leftarrow$  **loyalty**  $\rightarrow$  **churn**

Loyal customers install the app *and* stay.  
The graph claims  $\text{app} \perp\!\!\!\perp \text{churn} \mid \text{loyalty}$ .

## DIRECT EDGE

**app**  $\rightarrow$  **churn**  $\leftarrow$  **loyalty**

The app itself keeps customers. App and churn stay dependent **given** loyalty.

# The Churn Shop, Drawn

App users churn less (Lecture 02: the gap is real). Two graphs explain it:

## FORK

**app**  $\leftarrow$  **loyalty**  $\rightarrow$  **churn**

Loyal customers install the app *and* stay.  
The graph claims  $\text{app} \perp\!\!\!\perp \text{churn} \mid \text{loyalty}$ .

## DIRECT EDGE

**app**  $\rightarrow$  **churn**  $\leftarrow$  **loyalty**

The app itself keeps customers. App and churn stay dependent **given** loyalty.

The graphs disagree on one checkable claim. Compare app users and non-users **of equal loyalty** (say, the same tenure): equal churn favours the fork, a remaining gap the direct edge.

# Module 4

---

## Markov Networks

**Guiding Question:** *Pixels in an image influence each other symmetrically — no pixel causes its neighbour. How do we model that?*

# Undirected Models

**Markov network (Markov random field).** An **undirected** graph in which the joint distribution factorises over the **cliques**  $c \in \mathcal{C}$  of the graph:

$$p(x_1, \dots, x_n) = \frac{1}{Z} \prod_{c \in \mathcal{C}} \phi_c(x_c), \quad Z = \sum_x \prod_c \phi_c(x_c)$$

The  $\phi_c \geq 0$  are **potentials** — arbitrary non-negative functions, not probabilities.  $Z$  is the normalising **partition function**.

When all cliques have size 2 the model is a **pairwise Markov network**.

# Directed vs. Undirected

## Belief network

Directed, acyclic. Factors are conditional probabilities. Natural for **causal or generative** stories; conditioning on a collider creates dependence.

## Markov network

Undirected. Factors are potentials with a global  $Z$ . Natural for **symmetric interactions** — pixels, atoms, neighbours in a lattice.

# Marginalising and Conditioning Behave Oppositely

## Marginalising over $C$

makes  $A$  and  $B$  **dependent** — the sum over all states of  $c$  couples the potentials.

## Conditioning on $C$

In the chain  $A - C - B$ , a fixed  $c$  is a constant in the potentials, so the factor splits and  $A \perp\!\!\!\perp B \mid C$ .

*This holds because  $C$  separates them — not for every  $C$ .*

# Markov Properties and the Separation Test

For undirected models, independence is plain graph separation: if every path from  $X$  to  $Y$  passes through  $Z$ , then  $X \perp\!\!\!\perp Y \mid Z$ .

For a **directed** model the same question is answered by converting the graph:

## 1. Ancestral graph

Keep  $X \cup Y \cup Z$  and their ancestors; delete all other nodes and their edges.

## 2. Moralisation

Connect any two nodes that share a child, then drop all arrow heads.

## 3. Separation

Remove the nodes in  $Z$  and their edges. If no path joins  $X$  to  $Y$ , then  $X \perp\!\!\!\perp Y \mid Z$ .

# Two Examples

## Boltzmann machine

A pairwise Markov network over binary units with weights on the edges; the restricted variant (RBM) allows edges only between a visible and a hidden layer.

## Image cleanup

Pixels form a lattice: each potential prefers a pixel to agree with its neighbours and with the noisy observation. Inference denoises the image.

# Other Graphical Models

## Chain graphs

Combine directed and undirected edges in one model.

## Factor graphs

Make each factor an explicit node — mostly used **for inference**, since they state precisely which variables each factor touches.

Undirected models trade causal direction for symmetric interaction, and local normalisation for a global partition function. Separation in the (moralised) graph is the single test for conditional independence.

# Module 5

---

## Learning and Inference

**Guiding Question:** *We can now write the model down. Can we actually compute anything with it?*

# Three Questions You Can Ask a Model

## **Infer unobserved variables**

Compute the posterior given evidence — probabilistic inference.

## **Learn the parameters**

Estimate the factors' distributions from data.

## **Learn the structure**

Discover which dependencies exist at all.

With complete data, learning the parameters is the easiest of the three — it is counting. Learning the structure is the hardest.

# Inference Is Hard in General

Exact inference in belief networks is **NP-hard**, and so is approximate inference with a guaranteed error bound. Whether a particular model is tractable depends on its structure.

## Exact inference

Variable elimination, junction trees,  
message passing.  
Efficient when the graph is a tree or has  
low treewidth.

## Approximate inference

Sampling (MCMC, importance sampling)  
and variational methods.  
Trade the guarantee for tractability.

# Learning the Parameters

With **fully observed** data, parameter learning is counting: each conditional table is estimated from the frequencies of its variable and parents.

With **hidden variables**, the counts are unavailable and one alternates between inferring the hidden values and re-estimating the parameters (expectation–maximisation).

The estimates are only as good as the sample supporting each table row. A parent combination that never occurs in the data leaves its row undetermined — which is another argument for keeping tables small.

# Takeaway

Graphical models make the joint distribution **writable**; whether they make it **queryable** depends on the graph's structure. Parameters come from counting where data is complete, and from iterating where it is not.